

Когда пользователь — не человек

Тестирование МСП-серверов

Два мира API: у привычного и у агентского – **разные контракты**

REST / ПРИВЫЧНЫЙ МИР

- Контракт = OpenAPI-схема
- Description – декорация: перепишите как угодно, ни один клиент не сломается
- Потребитель – код
- Тест проверяет **код** – вердикт **pass / fail**

МСП / МИР АГЕНТОВ

- Контракт = схема + **description**
- Текст исполняем: формулировка меняет поведение системы
- Потребитель – модель
- Тест проверяет ещё и **текст** – вердикт **success rate**

Изменение description = breaking change, который не ловит ни один существующий тест

Значит, и тест другой – не pass/fail, а success rate

КЛАССИЧЕСКИЙ ТЕСТ

- Вход → выход **детерминирован**
- Вердикт: **pass** / **fail**
- Проверяем поведение **кода**
- Один прогон – и есть ответ

ТЕСТ АГЕНТНОГО ВЗАИМОДЕЙСТВИЯ

- Тот же вход → выход **варьируется**
- Вердикт: **success rate** – частота, не булево
- Проверяем ещё и **текст, который читает модель**
- Нужно N прогонов, чтобы измерить



Пример: один сервер, один запрос – **3 успеха из 10**. Откуда цифра – увидим дальше.

Тесты зелёные. Продукт не работает.

STAND · MVN TEST

```
server-java - mvn test

$ export JAVA_HOME=$(/usr/libexec/java_home -v 21)
$ mvn test
[INFO] Tests run: 11, Failures: 0 - TaskStoreTest
[INFO] Tests run: 3, Failures: 0 - v1 · описания как
javadoc
[INFO] Tests run: 7, Failures: 0 - v2 · описания для
модели
[INFO] Tests run: 5, Failures: 0 -
ToolsListSnapshotTest
[INFO] Tests run: 26, Failures: 0, Errors: 0,
Skipped: 0
[INFO] BUILD SUCCESS
Coverage (JaCoCo): 78% строк · 26 / 26 зелёные
```

Всё зелёное. Покрытие под 80%. А базовый сценарий агент не проходит.

ПРОФИЛЬ V1 · ПЛОХОЕ ОПИСАНИЕ

ПОЛЬЗОВАТЕЛЬ

Найди все просроченные задачи Алексея. Ответь списком id.

```
query_items({"filter": "assignee:Алексей AND overdue:true"})
```

```
← []
```

```
query_items({"filter": "assignee=Алексей overdue=true"})
```

```
← []
```

```
query_items({"filter": "assignee:Алексей"})
```

```
← []
```

... все 4 вызова вернули []

АГЕНТ

Просроченных задач не найдено.

Код **зелёный**. Тот же тул через агента – **пустой ответ**

ЮНИТ-ТЕСТ ТУЛА · TOOLBEHAVIOURTEST (V1)

```
// формат, который ждёт код → находит
@Test void findsTasksByKeywordFilter() {
    var r = tools.queryItems("Алексей overdue");
    assertThat(text(r)).contains("T-1","T-2","T-3"); // ✅ 3 задачи
}

// формат, что агент придумал по описанию
@Test void agentGuessedFilterFindsNothing() {
    var r = tools.queryItems("assignee=Алексей AND status=overdue");
    assertThat(isError(r)).isFalse();
    assertThat(text(r)).isEqualTo("[]"); // ✅ тоже зелёный!
}
```

Тот же тул, тот же стор. Верный формат находит три задачи – **код исправен**.
Формат агента даёт молчаливый [], и это **штатное поведение**.

ТОТ ЖЕ ТУЛ – ГЛАЗАМИ АГЕНТА

```
@McpTool(name="query_items",
    description="Queries items using filter") // ← всё, что видит агент
CallToolResult queryItems(String filter)
```

«Найди все просроченные задачи Алексея»

↓ формат не описан – агент подставляет произвольный

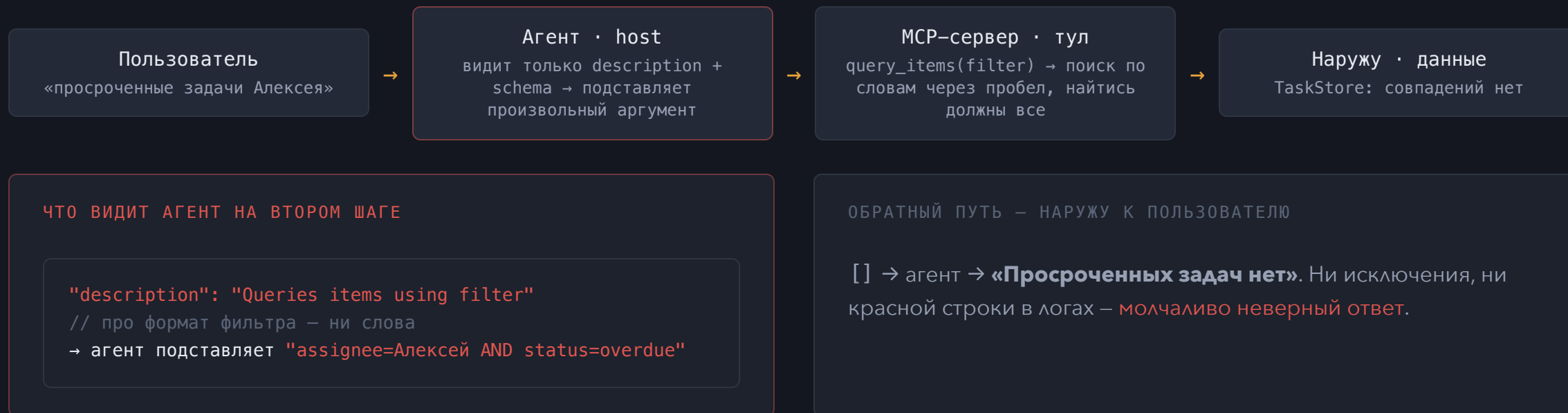
```
query_items(filter:
    "assignee=Алексей AND status=overdue")
```



[] → «Просроченных задач нет»

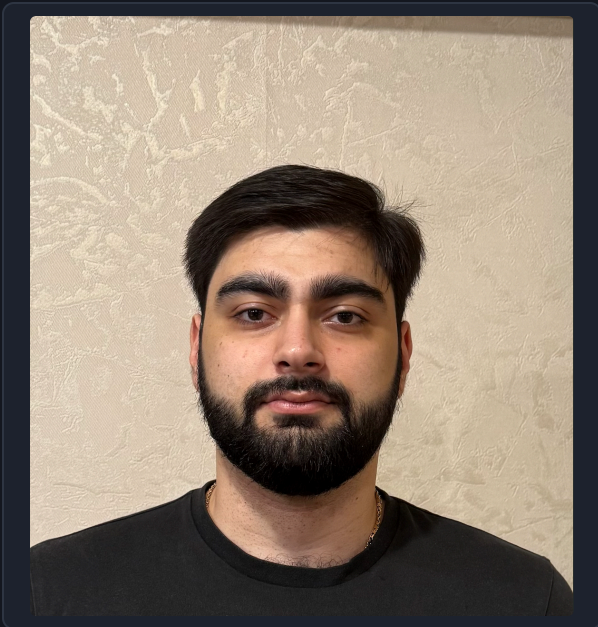
Юнит-тест проверяет **код** – он исправен. Сценарий ломает **описание**, а его этот тест не видит

Анатомия запроса: где именно **теряется смысл**



Между «что просил пользователь» и «что вызвать» стоит единственный ориентир — **description**. Плохой текст → неверный вызов

Кто я и о чём доклад



Левон Авакимянц,
Senior AQA, Сбер
Развиваю тестирование ИИ-агентов и
инструментов для агентов.

Основной пользователь
MCP-сервера – **ИИ-агент**.

Description тула – **исполняемая часть
контракта**.

Каждый ассистент к каждой системе – интеграции не масштабируются

5 АССИСТЕНТОВ, 10 СИСТЕМ

Каждую пару пишем отдельно – **50 интеграций**.

Новая система – ещё **+5**.

Общий протокол: каждый реализует его один раз – **15**.

Новая система – **+1**.

Формально: **N×M** → **N+M** – общий протокол вместо своей интеграции на каждую пару.

- Открытый стандарт (Anthropic, 2024)
- Поддержан: Claude, ChatGPT, Cursor, VS Code, JetBrains

КТО С КЕМ РАЗГОВАРИВАЕТ

Хост

Claude Desktop, Cursor, VS Code – модель живёт здесь. Пишете не вы

↕ JSON-RPC 2.0 · stdio или HTTP

MCP-сервер

это пишете вы – обычный процесс или веб-сервис

↕

Ваша система

БД · API · файлы

MCP-сервер – обычная детерминированная программа. Непредсказуемость этажом выше, в хосте

Три примитива. У tools инициатор – модель

ПРИМИТИВ	ЧТО ЭТО	КТО ИНИЦИИРУЕТ
tools	Действия: найти задачи, создать тикет, отправить письмо	Модель – сама
resources	Данные для контекста	Приложение
prompts	Готовые шаблоны	Пользователь

Модель сама решает: вызвать ли ваш код, какой тул, с какими аргументами

Ваш сервер для агента – **ВОТ ЭТОТ ТЕКСТ**. Другого интерфейса нет

ОДИН ТУЛ = ИМЯ + ОПИСАНИЕ + СХЕМА

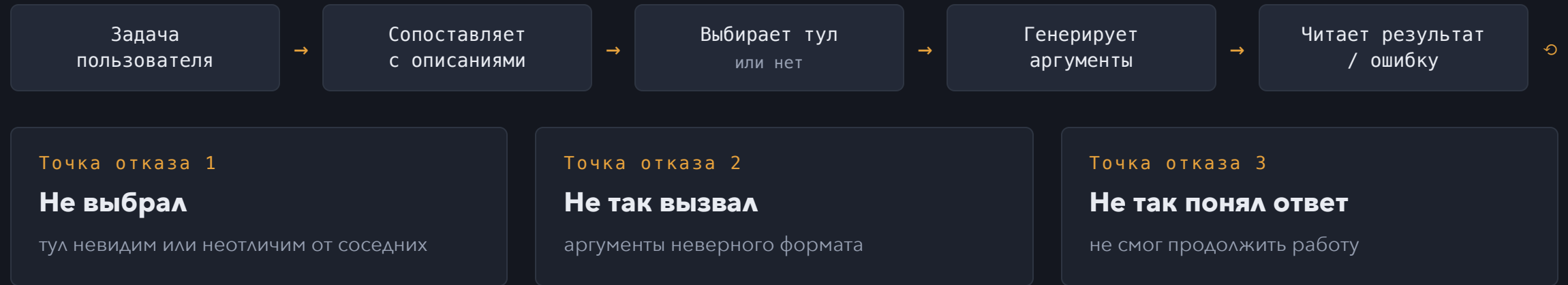
```
{
  "name": "query_items",
  "description": "Queries items using filter",
  "inputSchema": {
    "type": "object",
    "properties": {
      "filter": { "type": "string" }
    }
  }
}
```

...И ТАКИХ В КОНТЕКСТЕ МОДЕЛИ – ВСЕШ TOOLS/LIST

```
{
  "name": "query_items", "description": "Queries items using filter", "inputSchema": {...}
  "name": "create_item", "description": "Creates an item", "inputSchema": {"title": ..., "due_date": ...}
  "name": "update_item", "description": "Updates an item", "inputSchema": {"id": ..., "fields": ...}
  "name": "get_report", "description": "Gets report", "inputSchema": {"period": ...}
  "name": "delete_item", "description": "Deletes an item by id", "inputSchema": {"id": ...}
  "name": "list_users", "description": "Lists users", "inputSchema": {}
  ... и ещё N тулов от других подключённых серверов –
  все конкурируют за выбор модели в одном контексте
}
```

Модель не «подключается к API» – она **читает**. Заметьте, сколько здесь естественного языка.

Как агент решает – и где это ломается



Исправление — одна строка, и это **не код**

SNAPSHOT · TOOLS-V1.JSON → TOOLS-V1B.JSON

git diff · контракт tools/list

```
"name": "query_items",  
- "description": "Queries items using filter",  
+ "description": "Searches tasks. The filter  
+ is a set of space-separated keywords  
+ matched against task title, assignee and  
+ status; a task matches when every keyword  
+ matches. Statuses: open, in_progress,  
+ done, overdue. Example: \"Алексей overdue\"  
+ finds overdue tasks assigned to Алексей.",  
"inputSchema": { ... }
```

Ровно одна строка описания. Схемы, имена, остальные тулы — не тронуты. Логика поиска та же.

ПРОФИЛЬ V1B · +ОДНА СТРОКА ОПИСАНИЯ

ПОЛЬЗОВАТЕЛЬ

Найди все просроченные задачи Алексея. Ответь списком id.

`query_items({"filter": "Алексей overdue"})`

← [{"id": "T-1", "title": "Согласовать договор с Airtech", "assignee": "Алексей", "status": "overdue", "dueDate": "2026-07-18"}, {"id": "T-2", "title": "Об ...

АГЕНТ

Просроченные задачи Алексея: T-1, T-2, T-3

PR на одну строку · ревью за 10 секунд · **success rate 3/10 → 10/10**, вызовов на задачу 2.7 → 1.0

Description отвечает на вопрос «когда», а не «что»

✗ КАК ПИШЕТ РАЗРАБОТЧИК — ЧТО ДЕЛАЕТ ВНУТРИ

```
"Queries items using filter"
```

✓ КАК ЧИТАЕТ АГЕНТ — КОГДА ЗВАТЬ + ПРИМЕР

```
"Search tasks by assignee and/or status.  
Use this to find, count or list tasks  
(e.g. all overdue tasks for a person)."
```

Первое предложение — сценарий использования. Пример — похожий на реальные запросы пользователей: слово **overdue** связало запрос про «просроченные» с тулом.

Тулы конкурируют: **разграничивать**, а не множить

ПЕРЕКРЁСТНЫЕ ГРАНИЦЫ В ОПИСАНИЯХ

```
search_tasks:
    "...For aggregated statistics
    use get_task_report."

get_task_report:
    "...For finding individual
    tasks use search_tasks."
```

- Агент выбирает не «подходит ли тул», а «какой из 40 лучше»
- Каждый тул занимает контекст; после ~20-30 качество выбора падает
- Иногда решение — не улучшить описание, а **объединить или убрать** тулы

⚠ Раз текст исполняем — его можно атаковать: description — ещё и attack surface (tool poisoning). Ревью description = ещё и security-ревью.

Схема, ошибки, результат — тоже промпт

✗ «УГАДАЙ ФОРМАТ САМ» → ✓ СХЕМА СУЖАЕТ ОШИБКИ

```
query_items(filter: string)

search_tasks(
  assignee?: string,
  status?: "open"|"in_progress"|"overdue"|"done",
  due_before?: string // ISO, e.g. 2026-07-15
)
```

enum вместо строки — модель физически не пришлёт статус вне списка.
Дефолты опишите словами.

ОШИБКА — TOOL-LEVEL, НЕ PROTOCOL-LEVEL

protocol-level — «сервер сломался», агент видит сбой. **tool-level** — текст для модели: сюда валидацию, с форматом и примером → агент исправится сам.

РЕЗУЛЬТАТ — МАТЕРИАЛ ДЛЯ ШАГА

Не дампы на 40 полей с id и null'ами, а то, из чего делается следующий шаг диалога.

Всё неописанное агент домыслит сам. Ошибку и результат он **читает** — пишите их для него

Лучшая ошибка — та, которой не случилось

СОЗДАТЬ ЗАДАЧУ · «СРОК 5 АВГУСТА 2026 К 18:00»

v1 · «Error: Invalid date»

```
create_item(due_date:
"2026-08-05T18:00")
```

✗ Error: Invalid date

↓ повтор

```
create_item(due_date:
"2026-08-05")
```

✓ создано (T-9)

вызовов: 2

v1err · ошибка с форматом

```
create_item(due_date:
"2026-08-05T18:00")
```

✗ Invalid due_date: expected ISO date like 2026-07-28, got "2026-08-05T18:00". Today is 2026-07-23

↓ повтор

```
create_item(due_date:
"2026-08-05")
```

✓ создано (T-9)

вызовов: 2

v2 · формат в описании

```
create_task(due_date:
"2026-08-05")
```

✓ создано (T-9)

вызовов: 1

Один запрос, три контракта. Хорошая ошибка сокращает перебор; формат в описании убирает ошибку вовсе.

ВЫЗОВОВ ДО УСПЕХА · SONNET ×10

v1 **3.4** · v1err **2.0** · v2 **1.0** · успех везде 10/10, разница целиком в цене

КОНТРПРИМЕР

Пустой список — не ошибка. Там хороший текст ошибки не дал **ничего**: 3/10 и на v1, и на v1err.

МСР-сервер трекера задач: **какие тулы он предоставляет**

ЧТО ЭТО

- In-memory трекер задач · Java + Spring Boot + Spring AI, транспорт stdio
- Сид: **8 задач**, у Алексея **3 просроченные** (T-1...T-3) – тот самый сценарий из открытия
- Один сервер, **4 профиля контракта** (v1 · v1err · v1b · v2): меняется только текст описаний, логика одна

В открытии вы видели v1-версию первого тула – `query_items(filter)`. Справа он же в контракте v2 (для модели), `search_tasks`.

ТУЛЫ, КОТОРЫЕ СЕРВЕР ОТДАЁТ АГЕНТУ (V2)

ТУЛ	ЧТО ДЕЛАЕТ
<code>search_tasks</code> assignee? · status? · due_before?	Найти, сосчитать или перечислить задачи
<code>create_task</code> title · assignee? · due_date?	Создать задачу
<code>update_task</code> id · title? · status? · ...	Изменить задачу
<code>get_task_report</code> без параметров	Сводка по статусам

`status` – enum: `open` · `in_progress` · `overdue` · `done`

Ровно эти четыре действия агент выбирает по описаниям – на них и ставим все замеры

Пирамида тестов MCP-сервера – четыре уровня

УРОВЕНЬ	ЧТО ПРОВЕРЯЕТ	ВЕРДИКТ
TaskStoreTest · юнит	Доменная логика: поиск, создание, отчёт	детерминирован
ToolBehaviourTest · поведение тула	Что получает агент : форма ответа и тексты ошибок	детерминирован
ToolsListSnapshotTest · снимок	Контракт <code>tools/list</code> : имена, описания, схемы	детерминирован
eval-харнесс · сценарии	Задача агентом целиком, success rate	вероятностный

Три нижних уровня – как у любого сервиса, в CI на каждый коммит. Новое: под тест попал **текст** – описания и ошибки стали функциональными (подсвечены)

Текст контракта — **под обычный тест**, без участия агента

ПОВЕДЕНИЕ ТУЛА: АССЕРТ НА ТЕКСТ, КОТОРЫЙ ЧИТАЕТ МОДЕЛЬ

СНИМОК TOOLS/LIST: ПРАВКА ОПИСАНИЯ РОНЯЕТ СБОРКУ

```
// текст ошибки — часть контракта, пиним его
@Test void errorGuidesModel() {
    var r = tools.createTask("Отчёт", null, "завтра");
    assertThat(text(r)).matches(
        "Invalid due_date: expected ISO date "
        + "like \\d{4}-\\d{2}-\\d{2}, got \\\"завтра\\\"...");
}
// «Error: Invalid date» тест бы не принял
```

Форма ответа, пустой результат, тексты ошибок — детерминированы.

Обычный юнит-тест.

РУБЕЖ 1 · SNAPSHOT-ТЕСТ НА TOOLS/LIST

ToolsListSnapshotTest.java

```
@Test
@DisplayName("контракт v1 совпадает с эталоном")
void v1ContractMatchesSnapshot() {
    assertContract("tools-v1.json", new LegacyTools(new TaskStore()));
}
```

```
// сепар in-memory → tools/list → сверка с эталоном в VCS
void assertContract(String file, Object tools) {
    String actual = ToolsContractSnapshot.render(tools);
    String expected = ToolsContractSnapshot.read(file);
    assertThat(actual).isEqualTo(expected);
}
```

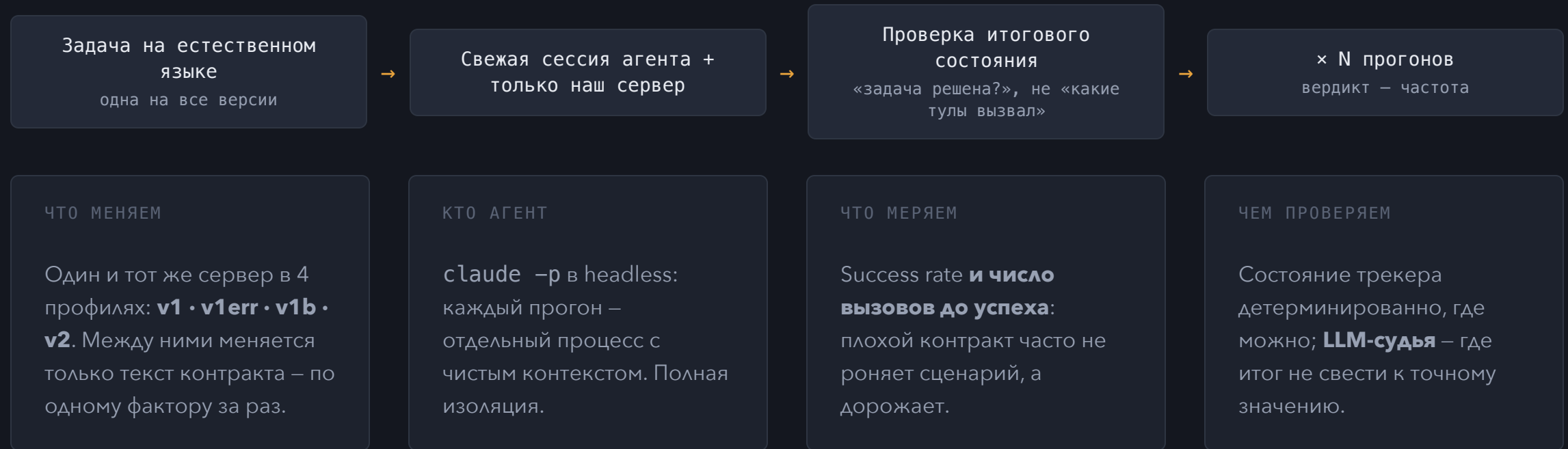
● mvn test — «причесали» описание перед релизом

```
FAILURE! v1ContractMatchesSnapshot
[контракт tools/list изменился; проверьте дифф и обновите эталон]
- "Queries items using filter"
+ "Search tasks by filter"
[ERROR] Tests run: 1, Failures: 1
```

Любая правка текста контракта роняет сборку и попадает в ревью отдельным диффом — а не в прод незаметно.

Оба детерминированы и гоняются в CI на каждый коммит. Ловят «безобидную» правку текста **до** релиза

Как собиралась статистика: один сервер, четыре контракта



Две метрики, и обе не «pass/fail» – как читать таблицу

МЕТРИКА 1 · SUCCESS RATE

N / 10

Один сценарий прогоняем **10 раз**. Success rate – сколько прогонов дошли до верного результата.



на картинке – **3 из 10**

- Единица – **доля**, не булево: поведение агента вероятно
- 10/10 – стабильно · 3/10 – «иногда срабатывает»
- Приёмка по **порогу** (напр. $\geq 8/10$), а не по точному числу

МЕТРИКА 2 · ВЫЗОВОВ ДО УСПЕХА

1.0 ... 4.6

Среднее число обращений агента к тулу, пока задача не решена (по успешным прогонам). **Меньше – лучше.**

- 1.0 – верный вызов сразу (идеал)
- 4.6 – многократный перебор: дорого по токенам и времени
- Нужна, потому что success rate часто 10/10 у обоих контрактов – **разница прячется в цене**

Цвет в таблице: ■ идеал – 100% и один вызов · ■ провал или дорогой перебор. Обе метрики – для **конкретной модели** (в таблице их две: Sonnet и Haiku)

Субъективное «агент не понимает» стало метрикой – и выявило регресс

SONNET · 10 ПРОГОНОВ НА СЦЕНАРИЙ

СЦЕНАРИЙ	V1	V1ERR	V1B	V2
overdue-alexey	3/10	3/10	10/10	10/10
вызовов	2.7	2.3	1.0	1.0
create-date	10/10	10/10	10/10	10/10
вызовов	3.4	2.0	3.4	1.0
tasks-per-person	10/10	10/10	10/10	10/10
вызовов	3.2	3.2	4.6	1.0

NAIKU · ТЕ ЖЕ СЦЕНАРИИ

СЦЕНАРИЙ	V1	V1ERR	V1B	V2
overdue-alexey	8/10	9/10	10/10	10/10
вызовов	5.2	6.6	1.0	1.0
create-date	10/10	10/10	10/10	10/10
вызовов	2.2	2.0	2.7	1.0
tasks-per-person	9/10	10/10	10/10	10/10
вызовов	2.4	2.5	1.8	1.0

РЕГРЕСС ОТ УЛУЧШЕНИЯ

Правка v1→v1b подняла overdue-alexey с 3/10 до 10/10 – и сломала выбор тула в другом сценарии:

- v1 query_items x2 → **get_report**
- v1b query_items **x5**, отчёт не найден
- v2 **get_task_report** – один вызов

Snapshot-дифф показал бы здесь только улучшение. Регресс виден единственным способом – прогоном.

НАПОМИНАНИЕ · ЕДИНИЦЫ ИЗМЕРЕНИЯ – НА ПРЕДЫДУЩЕМ СЛАЙДЕ

N/10 – success rate · вызовов – среднее до успеха (меньше – лучше)

■ 100% и один вызов · ■ провал / дорогой перебор

Ограничения и стоимость eval'ов

EVAL'Ы НЕ ДАЮТ

- Детерминизма
- Скорости (наши 240 прогонов – ~15 минут)
- Бесплатности
- Стабильной базы сравнения: сменилась модель – сменились числа

КАК ПРИМЕНЯТЬ

- Порог (например, $\geq 8/10$) вместо точного числа
- Фиксированный набор задач
- Запускать на изменения description/схем и перед релизом – не на каждый коммит
- N=10 на сценарий – единицы долларов (\$4.90 за все наши прогоны)
- Фиксировать модель в прогоне и писать её рядом с числом
- Проверка – детерминированно; LLM-судья только там, где иначе нельзя

Это не замена вашей пирамиде тестов – это новый этаж над ней

Чек-лист description-ревью

- 01 Первое предложение – **когда** использовать, не что делает внутри
- 02 Пример, похожий на реальные запросы
- 03 Похожие тулы разграничены перекрёстно
- 04 Параметры: говорящие имена, enum вместо строк, формат + пример
- 05 Дефолты описаны словами
- 06 Ошибки – tool-level, с форматом и примером для самоисправления
- 07 Результат – материал для следующего шага, не дамп
- 08 Behaviour-тест на тексты ошибок и форму ответа
- 09 Snapshot-тест на tools/list
- 10 Eval на ключевые сценарии, порог по success rate



Репозиторий: обе версии сервера, тесты, eval-харнесс – каждую цифру можно воспроизвести.

Контакты докладчика

В API для людей текст описывал контракт.
В API для агентов текст – **и есть контракт.**